

Introduction to Intensive Programming in C++

Lecture 3: Complete search and backtracking

Elias TSIGARIDAS

École Polytechnique

Contents

Factorial (from previous lecture)

Complete search

Backtracking

Definition of the factorial

We can multiply two numbers
of bitsize b in $\mu(b) = O(b \lg b)$

#ops = $O(n)$
straightforward algorithm

$$n! = 1 \cdot 2 \cdot 3 \cdots (n-1) \cdot n$$

result $n!$
bsz of output : $O(\lg n!) \approx O(n \lg n)$

on our way to compute $n!$
we also compute $k!$, $\forall k \leq n$

```
factorial( int n )  
{  
    if ((n == 1) || (n == 0)) return 1;  
    return n * factorial(n-1);  
}
```

intermediate bsz is $O(k \lg k)$

each multiplication costs
 $\mu(k \lg k) = O(k \lg k \cdot \lg(k \lg k))$
 $= O(k \cdot \lg^2 k)$

- We cannot do better than $O(n \log(n))$.
- Complexity $O(n^2 \log^2(n))$.

Exponential in the input (what is the input?)

Q: Can we do better? Why?

$$\sum_k O(k \cdot \lg^2 k) = \dots = O(n^2 \lg^2 n)$$

Factorial divide & conquer

We can multiply two numbers
of bitsize b in
 $\mu(b) = O(b \lg b)$

$$n! = \Pi(1, n)$$

just denote like
the factorial

$$\Pi(i, j) = \Pi(i, t) \cdot \Pi(t+1, j)$$

$$t = \frac{i+j}{2}$$

```
prod( int i, int j ) {  
    if ( i == j ) { return j; }  
    return prod(i, i + (j-i)/2) * prod(i + (j-i)/2 + 1, j);  
}  
fac_n = prod(1, n);
```


Factorial divide & conquer

We can multiply two numbers
of bitsize b in
 $\mu(b) = O(b \lg b)$

$$n! = \Pi(1, n)$$

$$\Pi(i, j) = \Pi(i, t) \cdot \Pi(t+1, j)$$

$$t = \frac{i+j}{2}$$

straightforward bound

$$\Pi(i, j) = i(i+1)(i+2) \dots (j-1) \cdot j \leq j \dots j \leq j^{j-i+1}$$

$$\text{bsz of } \Pi(i, j) \text{ is } O((j-i) \cdot \lg j)$$

```
prod( int i, int j ) {  
    if ( i == j ) { return j; }  
    return prod(i, i + (j-i)/2) * prod(i + (j-i)/2 + 1, j);  
}  
fac_n = prod(1, n);
```

Factorial divide & conquer

We can multiply two numbers
of bitsize b in
 $\mu(b) = O(b \lg b)$

$$n! = \Pi(1, n)$$

$$\Pi(i, j) = \boxed{\Pi(i, t)} \cdot \boxed{\Pi(t+1, j)}$$

$$t = \frac{i+j}{2}$$

$$\Pi(i, j) = i(i+1)(i+2) \dots (j-1) \cdot j \leq j \dots j \leq j^{j-i+1}$$

$$\text{bsz of } \Pi(i, j) \text{ is } O((j-i) \lg j)$$

```
prod( int i, int j ) {
    if ( i == j ) { return j; }
    return prod(i, i + (j-i)/2) * prod(i + (j-i)/2 + 1, j);
}
fac_n = prod(1, n);
```

cost of multiplication is

$$\mu((j-i) \lg j) = O((j-i) \lg^2 j)$$

$$\Pi(i, j) = \Pi(i, t) \cdot \Pi(t+1, j)$$

$$T(i, j) = T(i, t) + T(t+1, j) + O((j-i) \lg^2(j-i))$$

$$= \dots = O((j-i) \lg^3 j)$$

$$\Pi(1, n)$$

$$T(1, n) = O(n \lg^3 n)$$

Compare with
 $O(n^2 \lg^2 n)$

Factorial divide & conquer

We can multiply two numbers
of bitsize b in
 $\mu(b) = O(b \lg b)$

$$n! = \Pi(1, n)$$

$$\Pi(i, j) = \boxed{\Pi(i, t)} \cdot \boxed{\Pi(t+1, j)}$$

$t = \frac{i+j}{2}$

$$\Pi(i, j) = i(i+1)(i+2) \dots (j-1) \cdot j \leq j \dots j \leq j^{j-i+1}$$

bsz of $\Pi(i, j)$ is $O((j-i) \lg j)$

```
prod( int i, int j ) {  
    if ( i == j ) { return j; }  
    return prod(i, i + (j-i)/2) * prod(i + (j-i)/2 + 1, j);  
}  
fac_n = prod(1, n);
```

cost of multiplication is

$$\mu((j-i) \lg j) = O((j-i) \lg^2 j)$$

$$\begin{aligned} \Pi(i, j) &= \Pi(i, t) \cdot \Pi(t+1, j) \\ T(i, j) &= T(i, t) + T(t+1, j) + O((j-i) \lg^2(j-i)) \\ &= \dots = O((j-i) \lg^3 j) \end{aligned}$$

$$\Pi(1, n): T(1, n) = O(n \lg^3 n)$$

Compare with
 $O(n^2 \lg^2 n)$

Q Can we do better? **YES**

$$O(n \lg^2 n \lg \lg n)$$

[Borwein 1985] using prime factorization of $n!$

A question on factorial

TQ

Given n , $k \leq n$, and prime p compute $n! \bmod p$ or $\binom{n}{k} \bmod p$.

Hint: Wilson's theorem [first proof due to Lagrange] [primality test]

$$\text{If } p \text{ is prime} \Leftrightarrow (p-1)! = -1 \bmod p$$

A question on factorial

If $n \geq p$ then $n! \equiv 0 \pmod{p}$

$$n < p \Leftrightarrow \exists k \in \mathbb{Z}_+$$

$$\text{st. } n+k=p$$

$$n = p-k$$

I will compute this $n! = (p-k)!$

TQ

Given n , $k \leq n$, and prime p compute $n! \pmod{p}$ or $\binom{n}{k} \pmod{p}$.

Hint: Wilson's theorem [first proof due to Lagrange] [primality test]

$$\text{If } p \text{ is prime} \Leftrightarrow (p-1)! = -1 \pmod{p}$$

A question on factorial

If $n \geq p$ then $n! \equiv 0 \pmod p$

$$n < p \Leftrightarrow \exists k \in \mathbb{Z}_+$$

$$\text{s.t. } n+k=p$$

$$n = p-k$$

I will compute this $n! = (p-k)!$

$$(p-1)! = -1 \pmod p$$

$$(p-2)! = (-1)(p-1)^{-1} \pmod p$$

$$(p-3)! = (-1)(p-1)^{-1}(p-2)^{-1} \pmod p$$

$\vdots$

$$(p-k)! = (-1)(p-1)^{-1}(p-2)^{-1} \dots (p-k+1)^{-1} \pmod p$$

$$\text{BUT } (p-k)^{-1} = (p-k)^{p-2} \pmod p \longleftrightarrow \text{fast exponentiation}$$

it suffices to compute

$$(p-k)^{-1} \pmod p$$

p is prime

$$a^{p-1} = 1 \pmod p$$

$$a^{-1} = a^{p-2} \pmod p$$

TQ

Given n , $k \leq n$, and prime p compute $n! \pmod p$ or $\binom{n}{k} \pmod p$.

Hint: Wilson's theorem [first proof due to Lagrange] [primality test]

$$\text{If } p \text{ is prime} \Leftrightarrow (p-1)! = -1 \pmod p$$

$$(p-2)!(p-1) = -1 \pmod p$$

$$(p-2)!(p-1)(p-1)^{-1} = (-1)(p-1)^{-1} \pmod p$$

$$(p-2)! = (-1)(p-1)^{-1} \pmod p$$

A question on factorial

If $n \geq p$ then $n! \equiv 0 \pmod p$

$$n < p \Leftrightarrow \exists k \in \mathbb{Z}_+$$

$$\text{s.t. } n+k=p$$

$$n = p-k$$

I will compute this $n! = (p-k)!$

$$(p-1)! = -1 \pmod p$$

$$(p-2)! = (-1)(p-1)^{-1} \pmod p$$

$$(p-3)! = (-1)(p-1)^{-1}(p-2)^{-1} \pmod p$$

$\vdots$

$$(p-k)! = (-1)(p-1)^{-1}(p-2)^{-1} \dots (p-k+1)^{-1} \pmod p$$

$$\text{BUT } (p-k)^{-1} = (p-k)^{p-2} \pmod p \longleftrightarrow \text{fast exponentiation}$$

it suffices to compute

$$(p-k)^{-1} \pmod p$$

p is prime

$$a^{p-1} = 1 \pmod p$$

$$a^{-1} = a^{p-2} \pmod p$$

TQ

Given n , $k \leq n$, and prime p compute $n! \pmod p$ or $\binom{n}{k} \pmod p$.

Hint: Wilson's theorem [first proof due to Lagrange] [primality test]

$$\text{If } p \text{ is prime} \Leftrightarrow (p-1)! = -1 \pmod p$$

Q: Can we do better?

$$n < p \text{ and } p \nmid n!$$

the probability that p divides $n!$

$$\# \text{ prime factors of } x \text{ is } O(\frac{x}{\ln x})$$

Factorial and polynomials

[Bostan, Gaudry, Schost]
use polynomials!

TQ

Given n , $k \leq n$, and prime p compute $n! \bmod p$ or $\binom{n}{k} \bmod p$.

poly F

$\deg F = m$

evaluate F at 1 point, it costs

$O(m)$ by Horner [Pan] optimal

multipoint evaluation: Evaluate at m points costs $O(m \lg^2 m)$

Factorial and polynomials

[Bostan, Gaudry, Schost]
use polynomials

TQ

Given n , $k \leq n$, and prime p compute $n! \bmod p$ or $\binom{n}{k} \bmod p$.

$$\exists m \text{ s.t. } n-1 = m^2 \Rightarrow m = \sqrt{n-1}$$

Consider the polynomial

$$F(x) = (x+1)(x+2) \dots (x+m)$$

evaluate F at

$$x = 0, m, 2m, \dots, (m-1)m$$

poly F $\deg = m$

evaluate F at 1 point, it costs

$O(m)$ by Horner [Pan] optimal

multipoint evaluation: Evaluate at m points costs $O(m \lg^2 m)$

Factorial and polynomials

[Bostan, Gaudry, Schost]
use polynomials

TQ

Given n , $k \leq n$, and prime p compute $n! \bmod p$ or $\binom{n}{k} \bmod p$.

$$O(n) = m^2$$

$$\exists m \text{ s.t. } n-1 = m^2 \Rightarrow m = \sqrt{n-1}$$

Consider the polynomial

$$F(x) = (x+1)(x+2) \dots (x+m)$$

evaluate f at

$$x = 0, m, 2m, \dots, (m-1)m$$

$$x=0 \quad F(0) = 1 \cdot 2 \cdot 3 \cdot \dots \cdot m$$

$$x=m \quad F(m) = (m+1)(m+2) \dots (m+m)^{2m}$$

$$x=2m \quad F(2m) = (2m+1)(2m+2) \dots (2m+m)^{3m}$$

$\vdots$

$$x=(m-1)m \quad F((m-1)m) = (m^2-m+1)(m^2-m+2) \dots (m^2-m+m)^{m^2}$$

$$n! = F(0) \cdot F(m) \cdot \dots \cdot F((m-1)m)$$

$$\text{Cost} : O(m \lg^2 m) = O(\sqrt{n} (\lg \sqrt{n})^2)$$

$$m = \sqrt{n} \\ O(\sqrt{n} \lg^2 n)$$

$$\text{poly } F, \deg F = m$$

evaluate F at 1 point, it costs

$O(m)$ by Horner [Pan] optimal

multipoint evaluation: Evaluate at m points costs $O(m \lg^2 m)$

Contents

Factorial (from previous lecture)

Complete search

Backtracking

Complete search problems

- There is a finite number of elements (e.g., states, objects, etc).
- Find one (or all) element that satisfies some constraints.
- *Go through all the elements and check if they satisfy the constraints*
Brute Force
- Usually not very efficient!!!
- Sometimes is the only solution.
- You should always consider this solution first!
- It is very very very useful for debugging!

Complete search

exhaustive search / brute force

- Common complete search (or enumeration) problems:
 - All permutations of n elements.
 - All subsets of a set of n elements.
 - All combinations of k elements out of n .
 - The N queens problem.
 - Sudoku
- How do we iterate (enumerate) the search space?
(construct a tree ? What kind of a tree?)

Permutations of n elements

```
void permutations(std::vector<int>& V, int i, int n) {
    if (i == n - 1) {
        std::for_each(V.begin(), V.end(), [](const auto &e) { std::cout << e << " "; });
        return;
    }
    for (int j = i; j < n; j++) {
        // swap int at position `i` with the int at the current position
        std::swap(V[i], V[j]);

        // advance to the next position
        permutations(V, i + 1, n);

        // backtrack (restore the vector to its original state)
        std::swap(V[i], V[j]);
    }
}

int main() {
    std::vector<int> V = {1, 2, 3};
    permutations(V, 0, V.size());
    return 0;
}
```

C++ STL

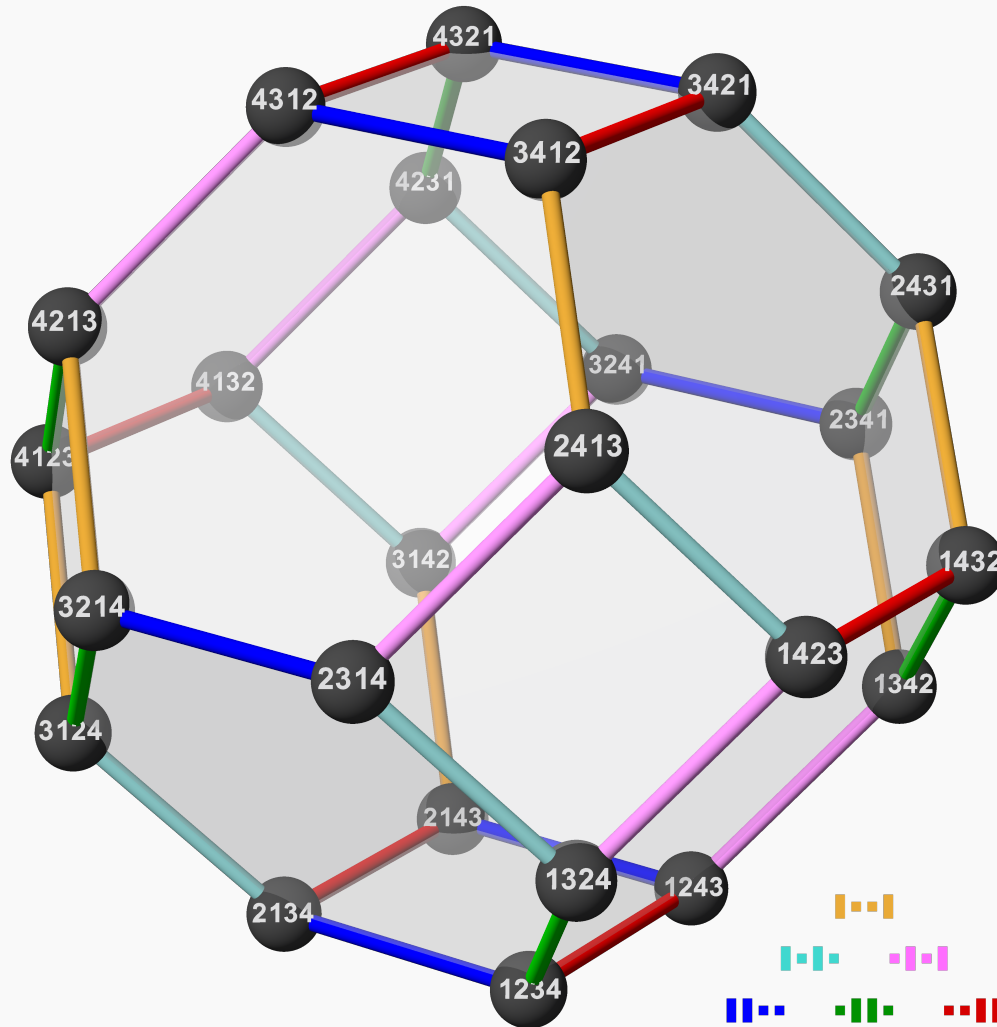
next_permutation
prev_permutation

lexicographical
ordering

TQ

How can we generate all the permutations using bitmasks ?

The geometry of permutations (permutohedron)



- It is of dim $n - 1$ and lives in the n -dim space.
- $\#\{\text{vertices}\} = n!$ and $\#\{\text{edges}\} = \frac{(n-1)n!}{2}$.
- Each vertex adjacent to $n - 1$ others. The length of the edges is $\sqrt{2}$.

Lehmer code Factorial number system

TQ

An algorithm to compute the **index** of a permutation when all permutations arranged in **lexicographical order** and its complexity.

		0	1	2	
		a	b	c	
0	a	b	c		
1	a	c	b		
2	b	a	c		
3	b	c	a		
4	c	a	b		
5	c	b	a		

fnCODE

$$\begin{array}{ccc}
 1 & 2 & 0 \\
 b & c & a \\
 1 & 1 & 0 \\
 1 \cdot 2! + 1 \cdot 1! + 0 \cdot 0! = 3
 \end{array}$$

$$\begin{array}{ccc}
 2 & 0 & 1 \\
 c & a & b \\
 2 & 0 & 0
 \end{array}$$

$$2 \cdot 2! + 0 \cdot 1! + 0 \cdot 0! = 4$$

$$2 = 1 \cdot 2! + 0 \cdot 1! + 0 \cdot 0!$$

1. Encode

↓ letter subtract from its value the #elems to the left that are smaller

2. Decode

↑ number add 1 to the numbers at the right that are not smaller

DECODE : given 4
4 = 2 · 2!

$$\begin{array}{ccc}
 2! & 1! & 0! \\
 2 & 0 & 0
 \end{array}$$

$$\begin{array}{ccc}
 2 & 0 & 1 \\
 c & a & b
 \end{array}$$

All subsets of n elements

- An integer in $[0..2^n - 1]$ represents a different subset of $\{1, 2, \dots, n\}$
- Generate all the integers

$\{1, 2\}$

```
int n = 5;
for (int subset = 0; subset < (1 << n); subset++) {
    for (int i = 0; i < n; i++) {
        if ((subset & (1 << i)) != 0) {
            std::cout << i + 1 << " ";
        }
    }
    std::cout << std::endl;
}
```

	1	2
$\emptyset$	0	0
$\{2\}$	0	1
$\{1\}$	1	0
$\{1, 2\}$	1	1

TQ

Why don't we do

```
for (int i = 0; i < 2^n; i++) { std::cout << i << std::endl; }
```

TQ: Subset sum

Find the subsets whose elements sum up to K .
(sort first! Why? Prune! How ?)

Some interesting problems

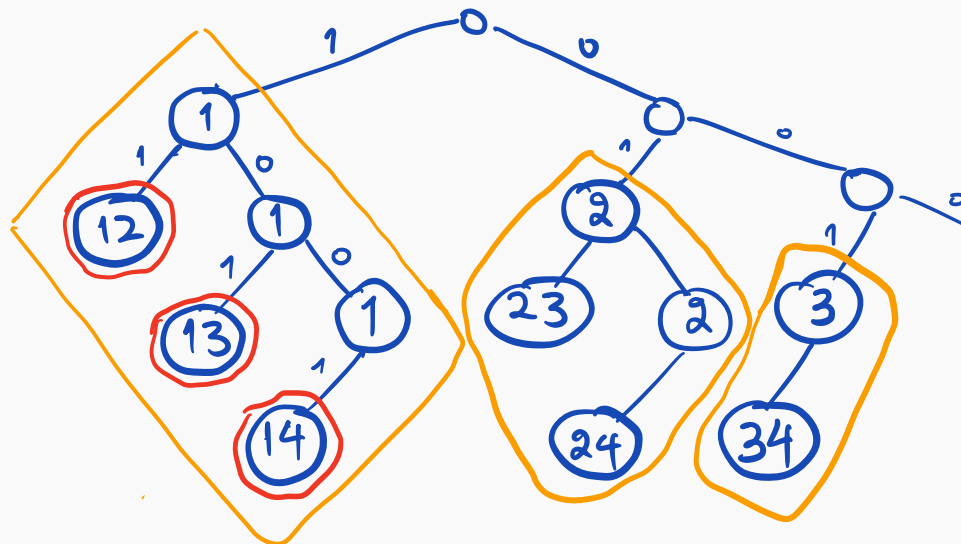
TQ (some subsets)

Given an array of n integers. Sort the subsets based on the sum of their elements. Find the (all) k -th subset from the list of sorted subsets.

TQ (all combinations $\binom{n}{k} \sim O(n^k)$)

How do we generate all combinations of k elements out of n ?

use inclusion-exclusion $O(2^n)$



ordering:

all the combinations containing 1 (as first elem)

all the combinations containing 2

$$O\left(\binom{n}{k} k\right)$$

at each height k
the decision is to
select or not
the k -th element

Contents

Factorial (from previous lecture)

Complete search

Backtracking

Backtracking

- Define **states**
 - We have one initial “empty” state.
 - Some states are partial.
 - Some states are complete (final).
 - **All** states are either **good** or **bad**.
- Define **transitions** from a state to possible next states
- States (nodes) and Transitions (edges) define a **tree**.
- Basic idea:
 1. Start with the empty state
 2. Use recursion to traverse all states using the transitions
 3. If the current state is invalid, then stop following its transitions.
 4. Process all complete states (these are the states we want)

like
turn-based games

Important property

We do not need to remember anything from the past (**memoryless**).
We depend on the current state only!

N-queens

The problem

Given N queens and an $N \times N$ chessboard, find **all/one** ways to place the N queens so that no queen can attack any other queen.

considered by
Gauss (of course)

[Franz Nauck]
1850

[Max Bezzel; 1848] for $N = 8$

[Francois-Joseph Eustache Lionnet; 1869] for general N

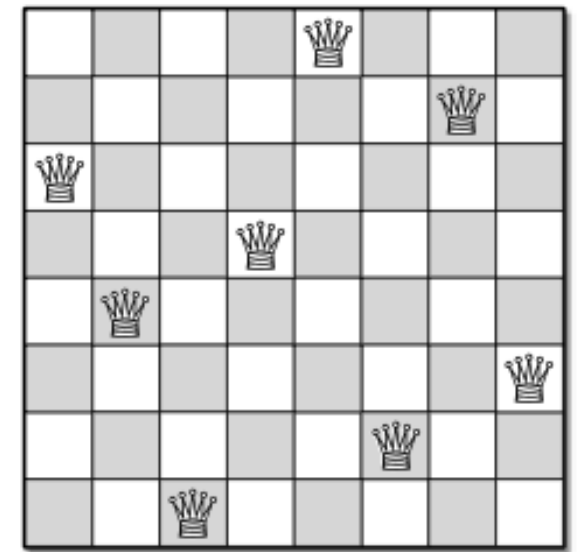
proposed the problem

[Bellavitis 1861]

- Place one queen at a time. Ensure there is conflict with queens already placed.
- How do we represent the state?
- Approach 1: (row,column) for each placed queen.
- Approach 2: A N by N boolean array
- Can we do better?

There is only one queen per row.

Store the column number!



The recursion tree (good and bad states) $N=4$

We always have a solution
for $N \geq 3$

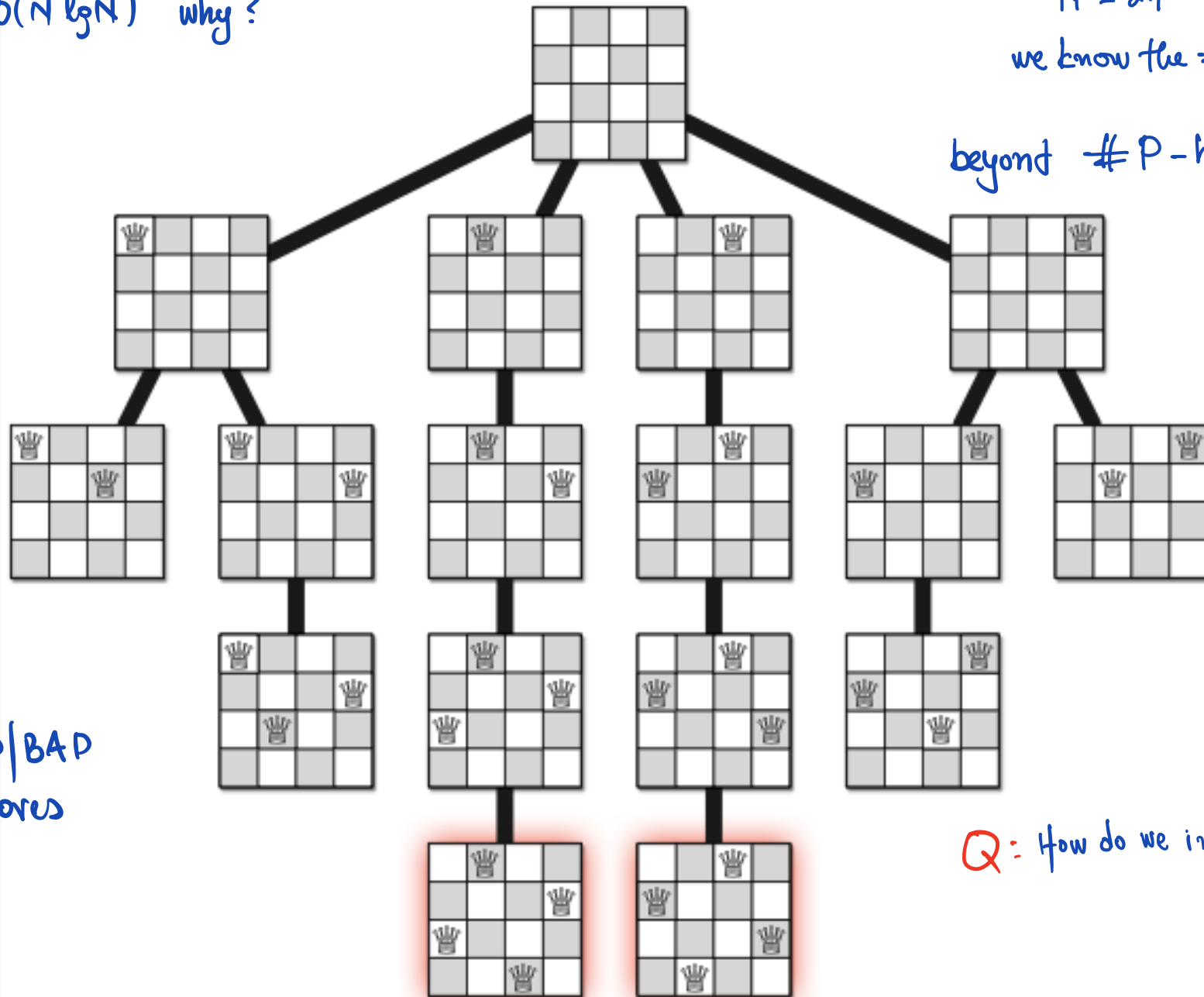
INPUT: the number N , $O(\lg N)$ bits

OUTPUT: $O(N \lg N)$ why?

$N \leq 27$

we know the #solutions

beyond # P-hard



sequence
of GOOD/BAD
moves

Q: How do we implement this?

A recursive algorithm for N -queens

PLACEQUEENS($Q[1..n], r$):

if $r = n + 1$

 print $Q[1..n]$

else

 for $j \leftarrow 1$ to n

$legal \leftarrow \text{TRUE}$

 for $i \leftarrow 1$ to $r - 1$

 if $(Q[i] = j)$ or $(Q[i] = j + r - i)$ or $(Q[i] = j - r + i)$

$legal \leftarrow \text{FALSE}$

 if $legal$

$Q[r] \leftarrow j$

 PLACEQUEENS($Q[1..n], r + 1$)

$\langle\langle \text{Recursion!} \rangle\rangle$

[Carl Friedrich Gauss; 1850]

[Emmanuel Laquière; 1882]

An easy question

TQ

What is the complexity of deciding if we can place N queens in $K \times L$ board?

An easy question

TQ

What is the complexity of deciding if we can place N queens in $K \times L$ board?

$O(1)$

Some more questions

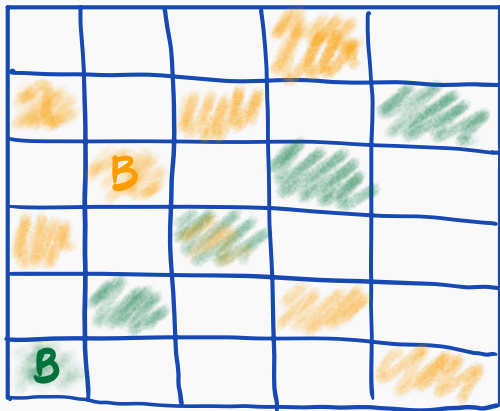
DFS = Depth First Search
(i.e. 1D arrays)

TQ

How many ways can we put down k mutually non-attacking bishops on an $n \times n$ board? Can we count the bishops without explicitly constructing every configuration?

TQ

Consider n bishops as (row, col) tuples on an $m \times m$ chessboard. Count the number of pairs of bishops that attack each other.

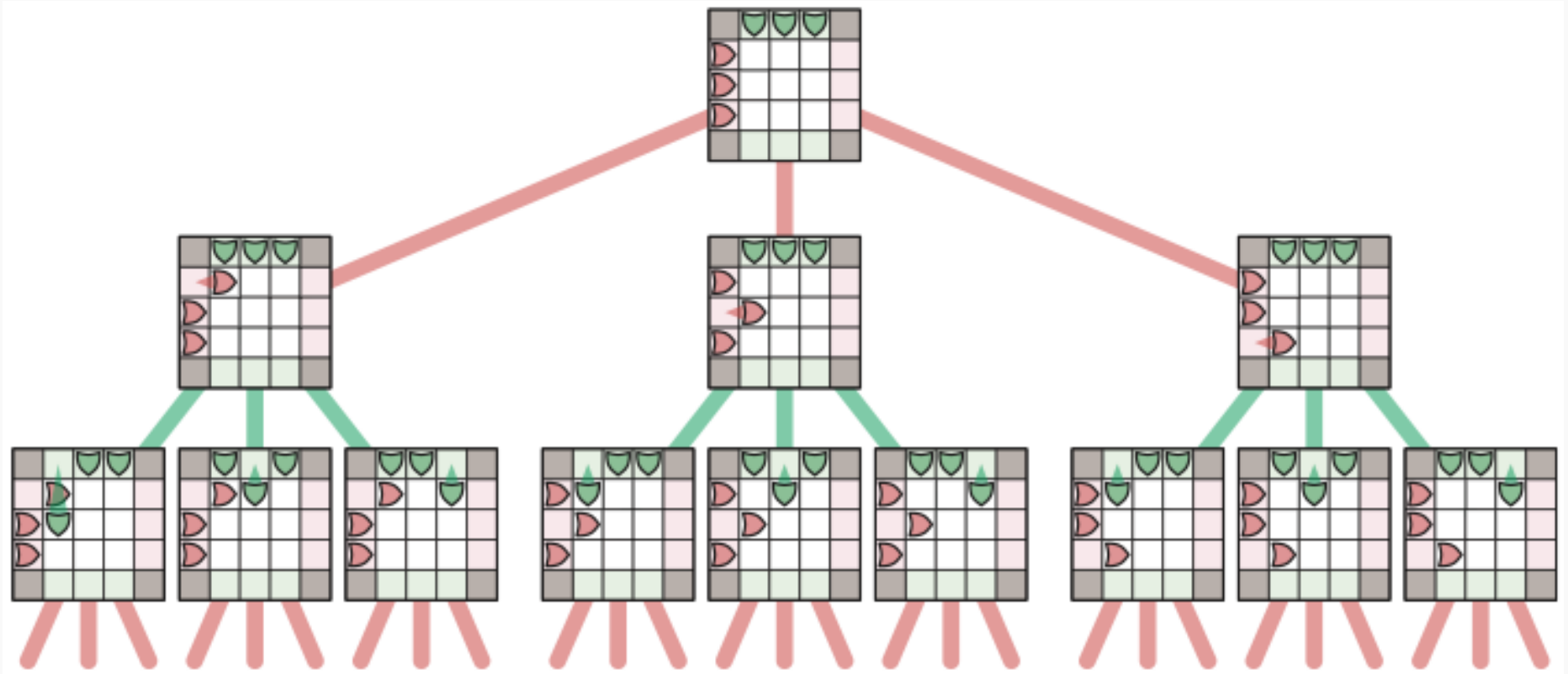


Backtracking & pruning

P Given one knight find
a way to visit all squares

Game trees (turn based games)

[Zermelo : 1913]
they can be infinite rounds !



Q: How do we implement this?

Text segmentation

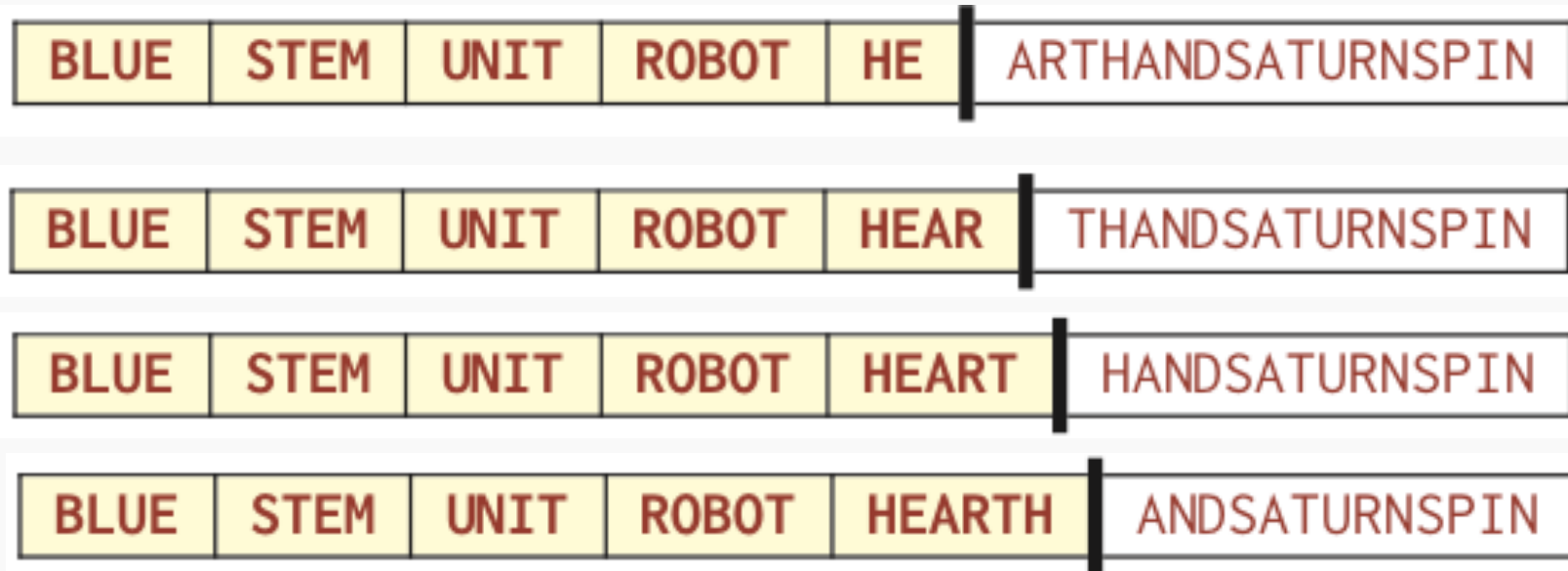
scriptio continua

PRIMVSDIGNITASINTAMTENVISCIANTIANONPOTEST
ESSERESENIMSVNTPARVAEPROPEINSINGVLISLITTERIS
ATQVEINTERPVNCTIONIBUSVERBORVMOCVPATAE

"First of all, dignity in such paltry knowledge is impossible; this is trivial stuff, mostly concerned with individual letters and the placement of points between words" [Cicero]

- Initially the words separated by dots
- Then with nothing (scriptio continua)
- Irish monks introduced spaces (around 8th century)
- Today, we have mixed system (hashtags, url, etc)

Split the text



SPLITTABLE($A[1..n]$):

if $n = 0$

return TRUE

for $i \leftarrow 1$ to n

if IsWORD($A[1..i]$)

if SPLITTABLE($A[i+1..n]$)

return TRUE

return FALSE

Straight forward Complexity
 $O(2^n)$

Using memoization
 $O(n^2)$

DP

An interesting problem

TQ (USACO 2015)

Given an $N \times N$ grid of fields ($2 \leq N \leq 18$), each labeled with a letter in the alphabet. Bessie walks from **upper-left** to **lower-right**, each step either to the **right** or **downward**. Compute the number of different palindromes she can form during her walk.

$$\# \text{paths} = (2N-2)! (N-1)! 2$$

INPUT

A	B	C	D
B	X	Z	X
C	D	X	B
W	C	B	A

Start (0,0)
End (N-1,N-1)

OUTPUT

4

PALINDROMES

ABCWCBA
ABCD CBA
ABXZXBA
ABXDXBA

Hint

Use backtracking and something else...

An interesting problem

TQ (USACO 2015)

Given an $N \times N$ grid of fields ($2 \leq N \leq 18$), each labeled with a letter in the alphabet. Bessie walks from **upper-left** to **lower-right**, each step either to the **right** or **downward**. Compute the number of different palindromes she can form during her walk.

$$\# \text{ paths} = (2N-2)! (N-1)! 2$$

INPUT

A	B	C	D
B	X	Z	X
C	D	X	B
W	C	B	A

Start (0,0)
End (N-1,N-1)

OUTPUT

4

PALINDROMES

ABCWCBA
ABCD CBA
ABXZXBA
ABXDXBA

Hint

Use backtracking and something else...

split the problem!

An interesting problem

TQ (USACO 2015)

Given an $N \times N$ grid of fields ($2 \leq N \leq 18$), each labeled with a letter in the alphabet. Bessie walks from **upper-left** to **lower-right**, each step either to the **right** or **downward**. Compute the number of different palindromes she can form during her walk.

$$\# \text{ paths} = (2N-2)! (N-1)! 2$$

INPUT



Start (0,0)
End (N-1,N-1)

OUTPUT

4

PALINDROMES

ABCWCBA

ABCD CBA

ABXZXBA

ABXDXBA

Hint

Use backtracking and something else... *split the problem!*

SUDOKU

What about SUDOKU ?

								8
2		1						
			9			6	7	3
				5				
9		7			3		4	1
				7				
	5					2	8	9
					5			
3	6			4				

7	9	6	5	3	4	1	2	8
2	3	1	7	8	6	9	5	4
5	4	8	9	1	2	6	7	3
6	2	3	4	5	1	8	9	7
9	8	7	6	2	3	5	4	1
4	1	5	8	7	9	3	6	2
1	5	4	3	6	7	2	8	9
8	7	2	1	9	5	4	3	6
3	6	9	2	4	8	7	1	5

It is an NP complete problem (for $n \times n$ grid)

Answer

We need **Algorithm X** to solve it. *Dancing links [knuth] aka Exact Cover*

*Q: How can we implement efficiently
a searching algorithm?*